CO5 – ASSESSMENT 1

CONSTRAINT-BASED PROBLEM SOLVING

OpenCV-Based System Design Solutions

Name	A. Archishman
Registration No.	192425341
Course Outcome	CO5 – Assessment 1
Topic	Constraint-Based Problem Solving (OpenCV)

Question 1: Real-Time Face Recognition for Campus Security

Problem Scenario: *A university wants to deploy a camera-based security system at the entrance that identifies registered students and staff in real time, under varying lighting, while remaining fast on low-cost hardware and minimizing false recognition.*

1. System Overview

The goal is a complete pipeline that takes live CCTV frames as input and outputs the identity of a person (or 'Unknown') with minimal latency. The pipeline is organized into five stages: image acquisition, preprocessing, face detection, feature extraction, and recognition, followed by optimization strategies that keep the whole system real time on modest hardware.

2. Image Acquisition

Frames are captured continuously from the CCTV/RTSP stream using OpenCV's VideoCapture. To avoid backlog when processing is slower than the camera frame rate, only the most recent frame is retained (a single-slot buffer updated by a dedicated capture thread), and processing is done on every nth frame (e.g., every 3rd frame) rather than every frame, since a person's identity does not change frame-to-frame.

3. Preprocessing

Colour space conversion: the BGR frame is converted to grayscale for detection speed, while a colour copy is retained for optional skin-tone/liveness checks.

Illumination normalization: Histogram Equalization or CLAHE (Contrast Limited Adaptive Histogram Equalization) is applied to compensate for uneven entrance lighting, backlighting, and shadows, which is essential because campus entrances have strong variation between daytime and artificial night lighting.

Noise reduction: a mild Gaussian blur removes sensor noise without erasing facial edges needed for detection.

Resizing: frames are downsized to a fixed working resolution (e.g., 640×480) before detection to bound computation cost, and the detected face region is later cropped from the original resolution for accurate feature extraction.

4. Face Detection

A Haar Cascade or an OpenCV DNN face detector (SSD-ResNet based res10_300x300_ssd) is used to locate face bounding boxes. The DNN detector is preferred for accuracy under pose and lighting variation, while Haar Cascade remains a lightweight fallback for very low-power

hardware since it runs purely on CPU with minimal memory. Detected regions are validated by a minimum size threshold to reject distant or partial faces that would produce unreliable features.

5. Feature Extraction

Two complementary approaches are combined depending on available compute:

Classical approach: detected faces are aligned using eye-landmark coordinates (via a facial landmark predictor), then represented using LBPH (Local Binary Pattern Histogram) features, which are illumination-robust and computationally cheap — well suited to low-cost hardware.

Deep-learning approach: where hardware allows, a pre-trained embedding network (e.g., OpenFace or a small FaceNet-style model run through OpenCV's DNN module) converts each aligned face into a 128-D embedding vector, giving much stronger discrimination between similar-looking individuals.

6. Recognition

For LBPH, OpenCV's built-in LBPHFaceRecognizer performs classification directly and returns a confidence/distance score. For embeddings, the extracted vector is compared against a pre-enrolled database of registered students/staff using cosine similarity or Euclidean distance, and a k-nearest-neighbour or SVM classifier assigns identity. A strict distance threshold is enforced: if the best match distance exceeds the threshold, the system reports 'Unknown' rather than forcing a match, which directly reduces false recognition.

7. Optimization for Real-Time Performance

- Frame skipping and region-of-interest tracking: once a face is detected, a lightweight tracker (KCF/CSRT) follows it across subsequent frames instead of re-running detection every frame.
- Multithreading: capture, detection, and recognition run on separate threads so I/O never blocks inference.
- Model choice: LBPH or MobileNet-based lightweight DNNs keep inference under 30 ms per face on CPU.
- Batching and caching: the enrolled-face database is loaded once into memory (a small vector index) rather than re-read per frame.
- Resolution scaling: detection runs on a downsized frame while recognition crops from full resolution for accuracy.

8. Justification of Accuracy and Real-Time Performance

Accuracy is achieved through illumination normalization (CLAHE) before detection, face alignment before feature extraction (removing pose-induced variance), and a distance-threshold rejection rule that suppresses false positives instead of forcing every detection into a known identity. Real-time performance is achieved by decoupling detection from tracking, running lightweight models suited to CPU hardware, and processing at a reduced but sufficient frame rate — together keeping end-to-end latency low while preserving recognition reliability.

Question 2: Intelligent Traffic Violation Detection System

Problem Scenario: *A traffic department wants roadside cameras to detect, track, and count vehicles crossing a line, flag abnormal movement, and display live counts/alerts, while handling shadows, noise, and dynamic backgrounds.*

1. System Overview

The pipeline covers acquisition, preprocessing, vehicle detection, tracking across frames, line-crossing counting, abnormal-motion detection, and visualization, with explicit handling of shadows and dynamic background clutter to minimize counting error.

2. Video Acquisition and Preprocessing

Acquisition: frames are read from the roadside RTSP/USB camera at a fixed rate; a region of interest (ROI) covering only the road area is cropped to reduce computation and ignore irrelevant background such as trees or sky.

Denoising: a Gaussian blur suppresses sensor and compression noise so it is not mistaken for motion.

Illumination handling: adaptive brightness correction is applied so day/night and weather changes do not trigger spurious foreground blobs.

3. Vehicle Detection

Two complementary techniques are used. Background subtraction (MOG2 or KNN background subtractor) extracts moving foreground blobs efficiently for continuous traffic flow. For higher accuracy in dense or overlapping traffic, a lightweight object-detection DNN (YOLO-tiny or MobileNet-SSD run through OpenCV's DNN module) classifies and localizes vehicles per frame, which is more robust to shadows and stationary vehicles than pure background subtraction.

4. Motion Analysis and Shadow/Noise Handling

- MOG2 with shadow detection enabled is used, and pixels marked as shadow (gray value in the foreground mask) are explicitly excluded from the vehicle blob before contour extraction.
- Morphological opening removes small noise blobs, and closing fills gaps inside a vehicle silhouette caused by uniform-coloured vehicle surfaces.
- A minimum contour-area threshold discards leaves, litter, or small dynamic background changes (e.g., swaying trees) from being counted as vehicles.

- A running background model is continuously updated at a slow learning rate so gradual lighting shifts (clouds, dusk) do not falsely trigger the whole frame as foreground.

5. Tracking Across Frames

Detected vehicle boxes are associated frame-to-frame using centroid tracking combined with a Kalman filter to predict motion, or with the SORT algorithm (a lightweight tracking-by-detection method) when a detector is used. Each vehicle is assigned a persistent ID, so a single vehicle is never counted twice even under brief occlusion.

6. Line-Crossing Counting

A virtual counting line is defined across the ROI. For each tracked vehicle, the centroid trajectory is checked against the line; a count increment occurs only on the frame where the centroid transitions from one side of the line to the other, and each tracked ID can only trigger one increment (state flag per ID), which prevents double counting from a temporarily flickering detection.

7. Abnormal Movement Detection

Trajectories from the tracker are analyzed for direction (wrong-way driving), abrupt deceleration/stopping in a live lane, or lane-line crossing, using simple rules on velocity vectors derived from consecutive centroids. Vehicles violating these rules are flagged and their bounding box is highlighted with an alert overlay.

8. Visualization and Output

Bounding boxes, IDs, the counting line, and a running count/alert panel are drawn on the frame with OpenCV drawing functions and displayed or streamed to a monitoring dashboard in real time.

9. Justification

Counting errors from shadows are minimized via MOG2's shadow flag and area filtering; dynamic background noise (trees, reflections) is filtered via ROI cropping, morphological cleanup, and area thresholds; multiple-vehicle confusion is resolved by persistent-ID tracking with a Kalman predictor, which maintains identity through brief occlusion so overlapping vehicles are not merged or double-counted.

Question 3: Automated Document Processing System

Problem Scenario: *A company receives thousands of scanned documents with blurred text, uneven illumination, noise, and varying fonts, and needs a batch pipeline that prepares them for accurate OCR.*

1. System Overview

The pipeline enhances each scanned image, removes noise/background artifacts, segments text regions, and geometrically corrects the page before handing it to an OCR engine (e.g., Tesseract), processing documents in batch for throughput.

2. Grayscale Conversion

Every scanned page is converted from colour/RGB to grayscale as the first step, since OCR only needs intensity/shape information; this immediately reduces data volume by two-thirds and removes colour-based noise (ink bleed-through, coloured stamps) that would otherwise confuse thresholding.

3. Thresholding

Adaptive thresholding (Gaussian or mean adaptive threshold) is applied instead of a single global threshold, because uneven illumination across a scanned page means no single brightness cutoff separates text from background everywhere; adaptive thresholding computes a local threshold per neighbourhood, producing a clean binary image even where the page is unevenly lit or partially shadowed by the scanner lid.

4. Noise Filtering

A median blur (rather than Gaussian) is applied before thresholding because it removes salt-and-pepper speckle noise common in scanned documents while preserving sharp text edges. After binarization, small isolated white/black speckles are removed by area-based connected-component filtering.

5. Morphological Operations

Opening: removes stray noise dots left after thresholding without damaging character strokes.

Closing: joins small gaps within broken characters caused by low print quality or blur, improving character connectivity for OCR.

Dilation with a horizontal kernel: merges characters within a word/line into connected blobs, which is used specifically to detect text-line regions during segmentation.

6. Geometric Correction

Skew is detected via the minimum-area bounding rectangle of text contours (or a Hough-line based estimate of dominant text-line angle) and corrected with an affine rotation, since even a small skew degrades OCR accuracy substantially. For photographed (non-flatbed) documents, a four-point perspective transform is applied after detecting the page boundary contour, correcting keystone distortion so the page appears as a flat rectangle.

7. Text Region Segmentation

After the dilation step merges characters into line-level blobs, contour detection extracts bounding boxes for each text line/paragraph block. Blocks are sorted top-to-bottom, left-to-right to preserve reading order, and each block is cropped and passed individually to the OCR engine, which improves accuracy over feeding the whole page at once, particularly for multi-column layouts.

8. Batch Processing

All stages are wrapped in a pipeline function applied over a queue/folder of documents using multiprocessing (one worker per CPU core), since each page is processed independently; results (cleaned images plus extracted text) are written incrementally so a single corrupt file does not stall the batch.

9. Justification of OCR Accuracy Gains

Grayscale and adaptive thresholding remove illumination bias that would otherwise be misread as missing ink; median filtering plus morphological opening/closing produce clean, well-formed character shapes; geometric correction ensures characters are upright and undistorted, which is critical because OCR engines are trained on horizontally aligned text; and segmentation preserves correct reading order — together these stages transform a noisy scan into an OCR-ready image with substantially higher recognition accuracy.

Question 4: Real-Time Multi-Person Tracking and Attendance System

Problem Scenario: *A college wants classroom CCTV to detect multiple faces, track and recognize each student, avoid duplicate marking, tolerate brief occlusion/head movement, and run continuously with low delay.*

1. System Overview

This system extends single-face recognition (as in Question 1) with multi-object tracking and per-identity state management, so that each recognized student is marked present exactly once per session, even with movement and temporary disappearance.

2. Multi-Face Detection

Each incoming frame is passed through a DNN-based face detector capable of returning multiple bounding boxes (e.g., OpenCV's SSD-ResNet face detector), which is preferred over Haar Cascade here because classrooms have many faces at varying angles and distances, and the DNN detector is markedly more robust to partial profile views.

3. Feature Extraction and Recognition

Each detected face is aligned and converted to an embedding vector (as in Question 1). The embedding is matched against the enrolled student database using nearest-neighbour distance with a rejection threshold, so unregistered or unclear faces are labelled 'Unknown' rather than mismatched.

4. Multi-Object Tracking for Identity Continuity

Recognition alone is not run on every frame for every face (too costly); instead each recognized face is assigned a tracker (using a centroid-distance association step similar to SORT/Deep SORT, or per-face KCF trackers) that follows it across subsequent frames using position and size continuity. Recognition is re-run periodically per tracked ID (e.g., every 1–2 seconds) to confirm identity, rather than every frame, which keeps the system real time while still verifying identity is stable.

5. Handling Head Movement and Temporary Disappearance

- Each tracked identity is given a short 'grace period' (a few seconds) during which a temporary loss of detection (occlusion, looking down) does not delete the track — the tracker predicts position from recent motion and re-associates the face when it reappears nearby.
- A tolerance margin on facial-landmark alignment allows moderate head rotation without triggering a false 'Unknown' before recognition is re-confirmed.

- If a track is lost beyond the grace period, it is closed, but the student's attendance record already logged remains valid — avoiding an accidental duplicate slot being opened for the same student re-entering the frame.

6. Preventing Duplicate Attendance

Attendance is logged in an in-memory set keyed by student ID for the current session. When a track is confirmed to match a student ID, the system checks whether that ID is already marked present; if so, no new record is written, regardless of how many times that student's face is re-detected or re-tracked during the session. This decouples 'detection events' (many per student) from 'attendance events' (exactly one per student per session).

7. Continuous Low-Delay Operation

- Detection runs on downscaled frames; recognition only on newly appeared or periodically re-checked tracks, not every face every frame.
- Multithreaded pipeline separates capture, detection/tracking, and recognition stages.
- A maximum-track cap and stale-track cleanup routine prevent unbounded memory growth over a long class session.

8. Justification

Identity is maintained across frames by combining lightweight per-frame tracking with periodic (not continuous) recognition, which is both computationally efficient and resistant to momentary occlusion. Duplicate attendance is prevented structurally, by session-level ID bookkeeping rather than relying on the tracker alone, so even if a track is lost and a new track for the same person forms later, the attendance system itself blocks a second entry.

Question 5: Smart Home Surveillance and Intrusion Detection

Problem Scenario: *A fixed home camera must detect significant motion, distinguish it from noise/small background changes, track intruders, flag unauthorized entry into a defined region, and run in real time on limited hardware while resisting false alarms from shadows, illumination change, and swaying trees.*

1. System Overview

The pipeline uses background subtraction as the core motion cue, followed by filtering, morphological cleanup, contour-based object detection, and simple tracking, with a defined 'restricted region' polygon used to decide when an alert should fire.

2. Background Subtraction

An adaptive background subtractor (MOG2, with shadow detection enabled) builds and continuously updates a statistical model of the static scene. Using an adaptive model (rather than a single reference frame) allows the background to slowly absorb gradual changes such as changing daylight, so they are not repeatedly flagged as motion.

3. Filtering and Noise Suppression

Pre-blur: a Gaussian blur before subtraction reduces sensor-noise-driven false foreground pixels.

Post-mask cleanup: morphological opening removes small speckle regions from the foreground mask (leaves, insects, sensor grain); morphological closing fills small holes inside a real moving object's silhouette so it is not split into multiple weak blobs.

4. Contour and Object Detection

Contours are extracted from the cleaned foreground mask, and a minimum bounding-box area threshold filters out small dynamic background elements (rustling leaves, small animals, camera-induced flicker) that do not meet the size expected of a person, while still catching a human-scale intruder.

5. Shadow and Illumination Handling

- MOG2's built-in shadow classification marks shadow pixels distinctly (grey value in the mask); these are removed before contour extraction so a person's shadow does not get treated as a second moving object or inflate the bounding box.
- A slow background-model learning rate absorbs gradual illumination drift (sun moving, lights turning on at dusk) instead of triggering a full-frame false alarm.

- For sudden global illumination change (e.g., light switched on), a frame-difference-magnitude check across the whole image is used to briefly pause alerting and let the background model re-adapt, avoiding a single spurious full-frame alarm.

6. Motion Analysis and Tracking

Surviving contours are tracked frame-to-frame with centroid tracking plus a Kalman filter to smooth trajectories, distinguishing a coherently moving object (an intruder walking) from a jittering pattern typical of trees or reflections, since real motion produces a smooth, persistent trajectory while background clutter produces short-lived, erratic detections that are filtered out by a minimum trajectory-length requirement.

7. Restricted-Region Intrusion Logic

A polygon representing the monitored entrance/restricted zone is defined once during setup. An alert is generated only when a tracked object's centroid enters this polygon and the track has persisted for a minimum number of consecutive frames (to avoid a single noisy detection triggering an alert), which keeps the system focused on genuine, sustained entries rather than momentary artifacts anywhere in the frame.

8. Real-Time Operation on Limited Hardware

- Background subtraction and morphology are computationally cheap and run comfortably on low-power CPUs (e.g., Raspberry Pi class hardware).
- Frame resolution is downscaled for detection while the full-resolution frame is retained only for saving alert snapshots.
- Processing every 2nd–3rd frame is sufficient for motion tracking purposes without perceptible loss of responsiveness.

9. Justification: Reducing False Alarms

False alarms from shadows are suppressed by MOG2 shadow detection; from gradual illumination change by adaptive background learning; from sudden illumination change by a temporary full-frame-difference guard; from trees/dynamic background by area filtering plus a minimum sustained-trajectory-length requirement before an object is considered a genuine track; and unauthorized-entry alerts are further restricted to sustained centroid presence inside the defined polygon, so brief or spurious detections anywhere in the frame do not trigger a false intrusion alert.